

William Anderson

Senior Software Engineer

Seminole, FL | williamandresondev@outlook.com | +1 (407) 801-1287 |
<https://www.linkedin.com/in/william-andreson-39b88b3aa>

Professional Summary

Senior Mobile Software Engineer with strong experience building consumer, healthcare, finance, logistics, and enterprise iOS applications using **Swift 5.0–5.9, Objective-C 2.0, UIKit, SwiftUI 2.0–4.0, Combine, Core Data, REST, and GraphQL**. Skilled in modernizing legacy Objective-C modules into Swift, resolving crash, memory, threading, and App Store issues, improving launch speed and API reliability, and delivering secure mobile features with flexible architecture across MVVM, VIPER, and clean modular design. Comfortable supporting release cycles from discovery to TestFlight, CI/CD, analytics, accessibility, and production monitoring for high-traffic iOS apps.

Technical Skills

- **Mobile: Swift 5.0–5.9, Objective-C 2.0, UIKit, SwiftUI 2.0–4.0, Combine, Core Data, AVFoundation, Core Location, MapKit, Push Notifications, Deep Linking, In-App Purchases**
- **Architecture: MVVM, MVC, VIPER, Coordinator Pattern, Clean Architecture, Dependency Injection, Protocol-Oriented Programming**
- **Networking: REST APIs, GraphQL, URLSession, Alamofire 4.x–5.x, OAuth 2.0, JWT, SSL Pinning**
- **Database & Storage: Core Data, SQLite, Realm 3.x–10.x, Keychain, UserDefaults, FileManager**
- **Testing: XCTest, XCUITest, Snapshot Testing, Unit Testing, UI Automation, Crash Reproduction**
- **DevOps & Tools: Xcode 8.x–15.x, TestFlight, App Store Connect, Fastlane 2.x, GitHub Actions, Bitrise, CocoaPods, Swift Package Manager**
- **Monitoring: Firebase Crashlytics, Sentry, App Analytics, Instruments, Memory Graph Debugger, Xcode Organizer**

Professional Experience

Viaro Networks Inc.

Senior Software Engineer | *January 2021 – December 2025*

- Built enterprise, healthcare, logistics, and financial mobile applications using **Swift 5.5–5.9, Objective-C 2.0, UIKit, SwiftUI 3.0–4.0, and iOS 15–17** for production iOS users.
- Modernized large Objective-C screens into **Swift 5.7** modules with MVVM and coordinator navigation, reducing feature delivery risk by **38%** while preserving stable legacy flows.
- Implemented a secure biometric login feature using **LocalAuthentication**, Keychain storage, OAuth 2.0 token refresh, and device-level fallback handling for regulated finance users.
- Resolved a critical crash caused by unsafe background-thread UI updates by moving state changes to the main actor and adding Crashlytics monitoring, lowering crash-free session issues by **42%**.
- Migrated networking from callback-heavy Objective-C services to **URLSession async/await** in **Swift 5.5**, improving readability and cutting duplicated retry logic across the app.
- Improved app launch performance by profiling with **Instruments**, lazy-loading heavy SDK initialization, and reducing cold-start time by **31%** on older iPhone devices.
- Built reusable SwiftUI dashboard widgets while keeping UIKit containers for older screens, allowing the team to introduce modern UI without rewriting the whole mobile platform.

- Fixed a deep-link routing issue where authenticated users landed on incorrect screens by centralizing route parsing, session checks, and fallback navigation inside the coordinator layer.
- Implemented push notification workflows for shipment alerts, appointment reminders, and payment status updates using APNs, Firebase Messaging, and server-driven payload validation.
- Improved offline support with **Core Data**, background sync queues, conflict handling, and retry policies, helping field users continue workflows during unstable network conditions.
- Created Fastlane and TestFlight release automation with build-number management, certificate validation, and environment-based configurations, reducing manual release errors by **35%**.
- Mentored mobile developers on **Swift 5.9**, Objective-C interoperability, memory debugging, XCTest coverage, and clean pull-request practices for maintainable iOS delivery.

VividCloud

Software Engineer | January 2016 – December 2020

- Developed customer-facing retail, media, and healthcare iOS applications using **Swift 3.0–5.3**, **Objective-C 2.0**, **UIKit**, **Core Data**, and **iOS 10–14**.
- Migrated legacy Objective-C view controllers into **Swift 4.x–5.x** services and view models, improving modularity while keeping stable UIKit screens in production.
- Implemented a real-time appointment booking feature with calendar selection, timezone-safe scheduling, REST API validation, and push reminders that reduced missed booking issues by **29%**.
- Fixed a memory leak caused by retain cycles in closures and delegates by applying weak references, deinit validation, and Xcode Memory Graph debugging across affected modules.
- Refactored a slow product listing flow by adding API pagination, image caching, and background decoding, improving scroll performance by **36%** on mid-range devices.
- Built secure payment and checkout screens with tokenized card handling, SSL pinning, Keychain storage, and defensive error states for failed authorization responses.
- Introduced **XCTest** coverage for authentication, profile update, and order workflows, helping the team catch regression issues before TestFlight delivery.
- Replaced several fragile storyboard transitions with coordinator-based navigation, making feature modules easier to test, reuse, and migrate into newer Swift components.
- Integrated Firebase Crashlytics and analytics events to identify frequent production failures, prioritize bug fixes, and measure mobile funnel completion rates.
- Updated older CocoaPods dependencies, resolved build conflicts during Xcode upgrades, and stabilized CI builds for multiple branded iOS applications.
- Collaborated with backend and QA teams to reproduce API mismatch bugs, define contract fixes, and release patched mobile versions with minimal user disruption.

Minimalist Moon

Software Developer | January 2013 – December 2015

- Built early-stage consumer, travel, and marketplace iOS applications using **Objective-C 2.0**, **UIKit**, **Xcode 5.x–7.x**, **iOS 7–9**, and Cocoa Touch.
- Developed reusable Objective-C components for login, product browsing, profile editing, image upload, and map-based discovery flows across multiple mobile clients.
- Implemented Auto Layout migrations from fixed-frame interfaces, improving screen compatibility across iPhone 4s, iPhone 5, iPhone 6, and early iPad layouts.
- Fixed a recurring crash caused by nil dictionary values from unstable API responses by adding defensive parsing, model validation, and clearer user-facing error states.
- Integrated **AFNetworking 2.x** for REST communication, request cancellation, timeout handling, and JSON serialization in mobile applications with unreliable network conditions.
- Improved local persistence with **Core Data** and SQLite-backed caching, allowing users to view recent content and partially complete forms without constant connectivity.

- Supported a gradual migration from manual dependency handling to **CocoaPods 0.33–1.0**, reducing setup problems and improving consistency across developer machines.
- Implemented image compression and upload retry logic, reducing failed media submissions by **27%** for users on slower cellular networks.
- Worked closely with designers to convert Photoshop-based mobile designs into UIKit screens with custom table views, collection views, and reusable cells.
- Debugged App Store review issues involving privacy descriptions, background modes, and device compatibility, helping releases move through approval more smoothly.

Microsoft

Software Developer Intern | *July 2012 – December 2012*

- Supported internal mobile prototype work using **Objective-C 2.0, UIKit, Cocoa Touch, Xcode 4.x–5.x**, and early **iOS 6–7** development practices.
- Built small iOS proof-of-concept screens for authentication, list rendering, local settings, and data entry using UIKit controllers and storyboard-based layouts.
- Assisted senior engineers with Objective-C debugging, memory warnings, delegate callbacks, and crash reproduction across different simulator and device configurations.
- Wrote basic unit tests with early XCTest workflows and documented manual QA steps for login, navigation, form validation, and offline behavior.
- Fixed UI alignment issues caused by fixed-size layouts by applying Auto Layout constraints and testing screens across multiple iPhone resolutions.
- Reviewed API response handling code and added safer null checks, error messages, and retry behavior for unstable development endpoints.
- Used Git branching, pull requests, and code reviews to learn production engineering discipline inside a larger enterprise software environment.
- Prepared internal documentation for build setup, provisioning profiles, simulator testing, and common troubleshooting steps for new mobile contributors.
- Gained practical experience with **Objective-C**, UIKit lifecycle management, App Store constraints, and collaborative mobile development standards.

Education

Florida State University

Bachelor's Degree in Computer Science | 2008 – 2012